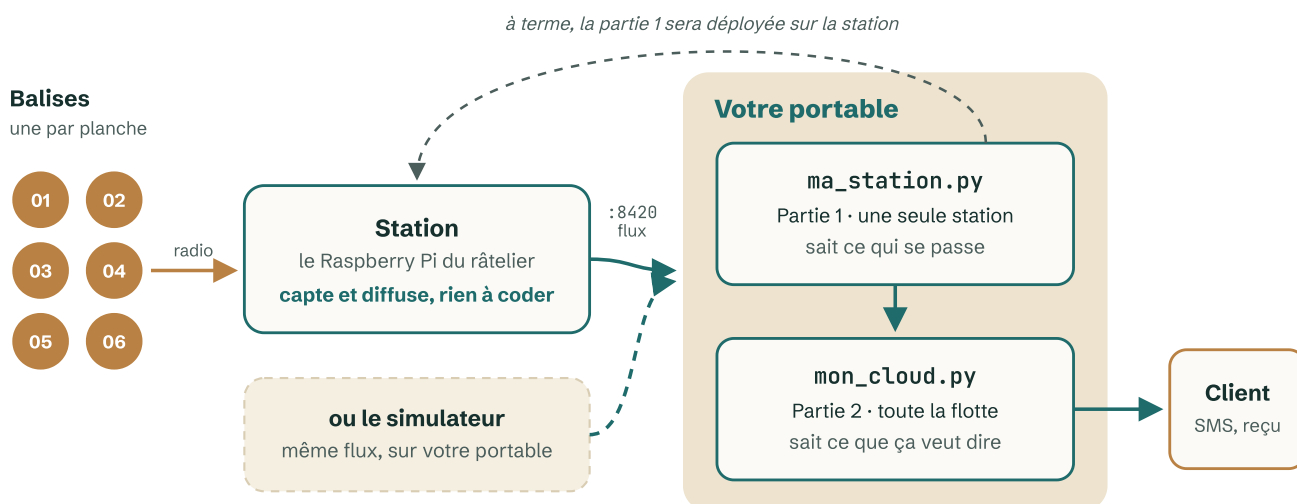


Kit hackathon

Des planches de surf en liège, en libre-service, payées à la minute. Chaque planche porte une balise radio, chaque râtelier une station qui l'écoute. Votre travail : transformer ce que les stations entendent en un parc de planches tenu juste, et en un service qui parle bien à ses clients.

Tout votre code tourne sur votre portable. Rien ne s'installe sur les stations. Une station ne fait que capter les balises et diffuser ce qu'elle entend. Vous l'écoutez à distance, ou vous lancez le simulateur, qui joue son rôle à l'identique. Même code, même format, dans les deux cas.

📄 Télécharger le kit (korko-kit.zip)



Aujourd'hui, les deux programmes tournent sur votre portable. À terme, la partie 1 sera déployée sur la station elle-même : écrivez-la comme si elle y vivait déjà.

Démarrer en cinq minutes

Rien à installer : Python 3 et sa bibliothèque standard suffisent.

1 Téléchargez le kit

📄 korko-kit.zip

Décompressez-le : vous obtenez un dossier `korko-kit` avec les cinq fichiers Python du kit. Ouvrez un terminal dans ce dossier ; toutes les commandes de cette page s'y lancent.

2 Rejoignez le wifi de la salle

Réseau

GL-SFT1200-3ae-5G

Mot de passe

goodlife

3 Vérifiez que tout est vert

```
python3 korko_test.py 192.168.8.100 --station A
python3 korko_test.py 192.168.8.101 --station B
python3 korko_test.py 192.168.8.102 --station C
```

Copier

Attendu, pour chacune : « Station conforme. » en une vingtaine de secondes. Sinon, levez la main.

4 Lancez les trois programmes, dans cet ordre

```
python3 korko_sim.py           # le terrain simulé → http://localhost:808
python3 cloud_exemple.py       # le cloud           → http://localhost:900
python3 station_exemple.py --source localhost:8420
```

Copier

Un terminal par ligne. Déplacez une planche sur la page du simulateur, et regardez le tableau de bord du cloud réagir.

Pour vérifier que les trois sont reliés : en haut de la page du simulateur, « 1 client(s) branché(s) » ; sur le tableau de bord du cloud, « A, dernier message à t = ... ». Si le cloud affiche « Aucune station branchée », c'est que la troisième commande ne tourne pas, ou qu'elle a été lancée sans `--source` : sans cette option, elle joue un scénario interne et n'écoute pas le simulateur.

Gardez aussi `korko.py` dans le même dossier. C'est la **bibliothèque**

commune du kit, à utiliser sans la modifier : elle lit le flux (simulateur, traces ou station réelle), appelle vos méthodes `observation` et `tic`, transmet vos décisions et les note. Vos programmes l'important ; vous n'avez jamais besoin de l'ouvrir.

5 Faites-en votre copie, et codez

```
cp station_exemple.py ma_station.py
cp cloud_exemple.py mon_cloud.py
```

[Copier](#)

Pour passer du simulateur à une vraie station, une seule chose change, toujours sur votre portable :

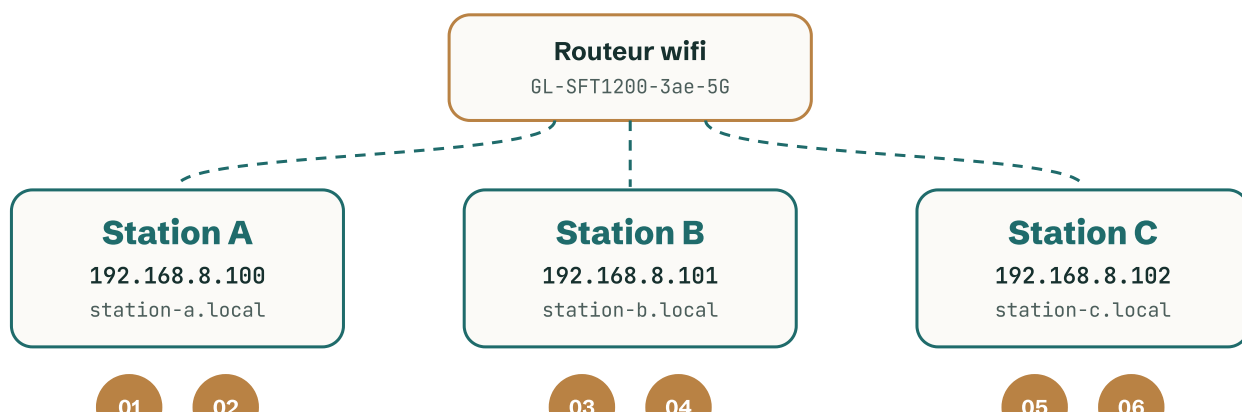
```
python3 ma_station.py --source 192.168.8.100:8420
```

[Copier](#)

La seule règle : n'appellez jamais `time.time()`. Des deux côtés, le temps arrive dans le champ `t` des messages. C'est ce qui permet de rejouer une journée en trente secondes, et de brancher le même code sur une station réelle sans rien y changer.

La maquette devant vous

Trois stations, six planches, un routeur. Le routeur joue le rôle du réseau 4G de la plage : il relie les stations entre elles, et à votre portable.



Chaque planche a une station de base, celle où elle se range.

Deux gestes suffisent

Pour simuler un départ, éloignez la planche à plus de quatre à cinq mètres du râtelier : au-delà, la station ne capte plus rien. **Pour un retour**, raccrochez-la. Une planche raccrochée à une autre station que la sienne est « étrangère ».

Sous Windows, utilisez les adresses : les noms en `.local` n'y fonctionnent pas. Adresse égarée ? `python3 korko_test.py --trouver` retrouve les trois stations.

Pourquoi deux parties, et pas une ?

Sur votre portable, tout pourrait tenir dans un seul programme. Mais sur la plage, les deux parties ne vivront pas au même endroit, et c'est le réseau qui les sépare.

La partie 1 est destinée à la station

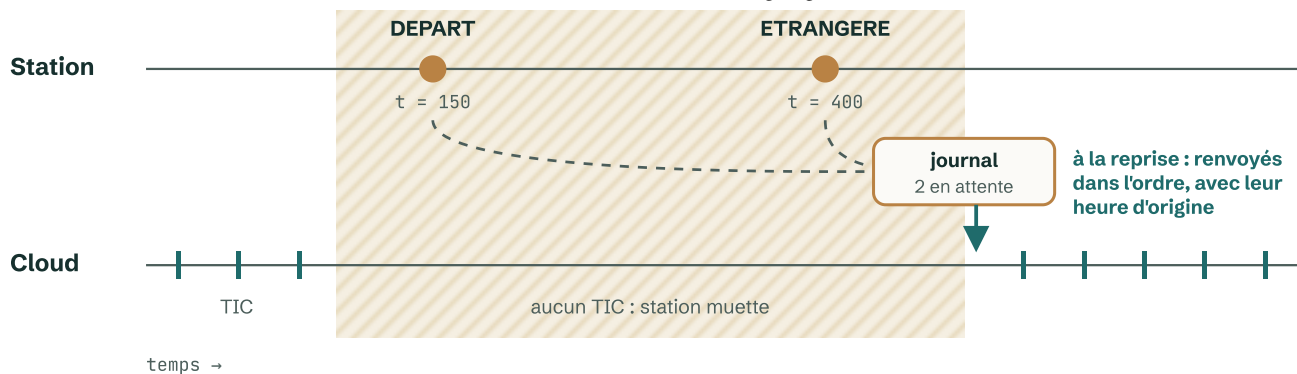
Il existe un risque que le réseau de la plage ne soit plus accessible, pour diverses raisons : 4G saturée un jour d'affluence, antenne en panne, alimentation coupée. Si cela arrivait, les surfeurs continueraient de partir et de rentrer. La station doit donc pouvoir savoir seule ce qui se passe à son râtelier, et en garder le journal. Quand le réseau redeviendrait accessible, elle renverrait ce journal dans l'ordre, et le cloud pourrait rattraper la situation.

La partie 2 vit au-delà du réseau

Elle gère la flotte entière : toutes les stations, les planches rendues ailleurs, le rééquilibrage, la surveillance du parc. Et elle porte tout ce qui touche à l'argent et aux personnes : comptes clients, sessions, facturation, cautions, SMS. Rien de cela ne peut vivre dans un râtelier.

En une phrase : **la station sait ce qui se passe ; le cloud sait ce que ça veut dire.**

Si le réseau devenait inaccessible



Ce qu'il se passerait en cas d'interruption du réseau : la station continue de détecter, garde ses événements, puis les renvoie dans l'ordre. Côté cloud, une station qui se tait n'est pas forcément un râtelier vide : c'est une station à surveiller.

Partie 1 — le code de la station

La station ne sait rien des clients, rien de l'argent, rien des autres stations. Elle sait une seule chose, et doit la savoir juste : **qui est là, qui vient de partir, qui vient de rentrer.**

Votre code tourne sur votre portable et lit le flux d'une station, réelle ou simulée. Une seule contrainte : écrivez-le comme s'il vivait dans le râtelier. Une seule station écoutée, aucune information venue d'ailleurs.

`korko.py` est la bibliothèque commune : elle fournit `Detecteur` et `Lancer`, utilisés dans le squelette ci-dessous. Importez-la, ne la modifiez pas.

Ce que vous recevez

Une ligne par paquet radio capté. Il n'y a pas de champ « présente » ni « partie » : c'est précisément ce que vous devez calculer.

```
{"t": 412.5, "station": "A", "balise": "korko-01", "rssi": -71}
```

[Copier](#)

Ce que vous produisez

Trois événements : `DEPART`, `RETOUR`, et `ETRANGERE` pour une planche d'une autre station raccrochée ici.

```
{"t": 447.0, "station": "A", "balise": "korko-01", "evenement": "DEPART"}
```

[Copier](#)

Le squelette

```
from korko import Detecteur, lancer
```

Copier

```
class MaStation(Detecteur):
```

```
    PERIODE_TIC = 1.0
```

```
    def observation(self, o):          # o.t, o.station, o.balise, o.rssi
        ...
```

```
    def tic(self, t):                  # même quand plus rien n'arrive
        ...
```

```
lancer(MaStation)
```

`observation` reçoit chaque paquet. `tic` est appelée à intervalle régulier, **y compris pendant les silences** : une planche partie n'émet plus rien vers la station, donc un départ ne se détecte que par une absence.

Quand vous décidez : `self.depart(balise, t, station)`, `self.retour(balise, t, station)`, `self.etrangere(balise, t, station)`. Sans le troisième argument, la station vaut « A ». Un scoreur compte automatiquement vos faux départs, vos retours manqués et votre latence.

Parler au cloud

`station_exemple.py` pousse chaque décision sur `http://localhost:9000/evenements`, et envoie un `TIC` à chaque seconde de flux, pour que le cloud ait une horloge même quand rien ne bouge.

Si le réseau devenait inaccessible, la station continuerait de détecter et garderait dans `self.journal` tout ce que le cloud n'a pas reçu. Elle retente à chaque `tic`, puis renvoie le journal dans l'ordre dès que la liaison est rétablie. Pour l'essayer : coupez le cloud une minute avec `Ctrl-C`, sortez une planche, relancez le cloud. Le départ arrive, avec son heure d'origine.

Les trois sources, le même code

```
python3 ma_station.py --sim --scenario journee      # simulateur, noté
python3 ma_station.py --rejeu traces/essai.ndjson  # traces enregistrées
python3 ma_station.py --source 192.168.8.100:8420  # station réelle
```

Copier

Utile : `--graine` pour rejouer à l'identique, `--chaos` pour des pertes de paquets et des trous réseau, `--duree` pour raccourcir une simulation, `-v` pour voir passer chaque paquet. En mode `--sim`, la simulation tourne d'une traite, sans attendre.

Ce que la radio va vous faire

- le RSSI bruite de quelques dB en permanence ;
- un corps mouillé devant la balise coûte une vingtaine de dB d'un coup, le cas qui casse les algorithmes naïfs ;
- une balise émet toutes les 400 ms sur la maquette, toutes les secondes dans le simulateur, et des paquets se perdent ;
- dans le simulateur : -67 dBm au râtelier, -80 à cinq mètres, -87 à neuf mètres, plus rien au-delà de trente. Le scénario `sable` y reproduit une planche posée à quelques mètres.

Sur la maquette, c'est plus simple : au-delà de quatre à cinq mètres, la station ne capte plus rien.

Les questions ouvertes, là où ça se joue

- Seuil sec, ou hystérésis à deux seuils ?
- Combien de secondes de silence avant de déclarer un départ ? Chaque seconde de trop est de la facturation perdue, chaque seconde de moins un risque de faux départ.
- Lisser le signal, ou traiter chaque paquet ?
- Distinguer « posée sur le sable à côté » de « partie à l'eau » ?
- Au démarrage, l'exemple suppose toutes ses planches au râtelier, et annonce partie celle qu'il n'entend pas. Est-ce la bonne hypothèse ?
- Le journal vit en mémoire : si le Pi redémarrait pendant une interruption, l'historique serait perdu. Un fichier suffirait.

Partie 2 — le cloud

Le cloud voit ce qu'aucune station ne voit : **toutes les stations à la fois, les clients, l'argent, et le temps long.** C'est un serveur. Ici, il tourne sur votre portable, exactement comme il tournerait ailleurs.

Le parcours complet en trois gestes

1 Un client arme sa session

Ouvrez `http://localhost:9000/arme?client=+33612&station=A` : le cloud lui désigne une planche.

2 Il part surfer, puis rentre

Dans le simulateur, ou sur la maquette, sortez cette planche du râtelier, puis raccrochez-la.

3 Il reçoit son reçu

Le SMS s'affiche dans le terminal du cloud. Le tableau de bord est sur `/`, l'état brut en JSON sur `/parc`.

Ce que vous recevez

Les stations poussent sur `POST /evenements`, une ligne JSON par événement : `DEPART`, `RETOUR`, `ETRANGERE`, et `TIC`, un battement d'horloge. **C'est sur le `TIC` que se vérifient les retards** : sans lui, une planche jamais rendue ne produirait aucun événement, donc aucune alerte. Là aussi, le temps, c'est le champ `t`.

Si une station perdait l'accès au réseau, elle vous renverrait d'un coup tout son journal une fois la liaison rétablie, avec les heures d'origine : un départ pourrait alors arriver avec vingt minutes de retard.

Les règles déjà écrites, toutes discutables

- la planche proposée est chez elle, au râtelier, et la moins sortie ;
- `DEPART` ouvre une session au nom du premier client armé sur la station ; si personne n'a armé, la planche passe « sortie sans client » et une alerte est levée ;
- `RETOUR` la ferme, facture 0,20 €/min, libère la caution, envoie le reçu ;
- `ETRANGERE` la ferme aussi, et signale une planche à rapatrier ;
- plafond dépassé : rappel SMS ; trois fois le plafond : planche réputée perdue, caution débitée.

`PLAFOND` vaut 600 secondes dans le fichier, et non trois heures, pour que le scénario

se déclenche pendant une démo. La fonction `sms()` imprime dans le terminal : ce que vous écrivez dedans compte plus que la plomberie, c'est toute la relation client.

Là où il y a tout à faire

- **Départs simultanés.** Deux clients arment en même temps : ils reçoivent la même suggestion, et le premier départ attrape le premier de la file. Comment lever le doute sans embêter personne ?
- **Rééquilibrage.** Quand rapatrier une planche rendue hors base, combien à la fois, dans quel ordre, à quel coût de tournée ?
- **Rotation.** « La moins sortie » ignore l'état réel, l'heure, la marée, l'affluence.
- **Confiance.** Un client qui rend toujours bien mérite-t-il une caution plus faible ? Une session plus longue ?
- **Surveillance.** Repérer une station muette, puis absorber son journal quand il arrive d'un coup, sans facturer deux fois ni déclarer perdue une planche dont la station était simplement injoignable.
- **Mémoire.** Tout est en RAM : si le cloud redémarrait, le parc serait amnésique.

Si vous préférez un vrai serveur

Déployez `mon_cloud.py` où vous voulez, ou réécrivez-le entièrement : FastAPI, Node, ce que vous voulez. Seul le contrat compte. Pour pointer la station dessus :

```
KORKO_CLOUD=https://mon-cloud.exemple/evenements python3 ma_station.py
```

[Copier](#)[Source](#)

Le simulateur

Il vous permet de tester des scénarios sans être physiquement devant les stations. Il fabrique le signal qu'une station recevrait de ses balises, et le diffuse exactement comme une vraie station, au même format. Votre code ne fait pas la différence.

C'est aussi l'endroit où l'on essaie ce que la maquette ne sait pas reproduire : une matinée entière en quelques secondes, un corps mouillé devant une balise, une planche posée sur le sable, une balise qui meurt, et le même tirage rejoué à l'identique pour comparer deux versions de votre code.

```
python3 korko_sim.py
```

```
# puis http://localhost:8080
```

[Copier](#)

Il fait deux choses en même temps : il affiche une page de contrôle sur le port 8080, pour vous, et il publie le signal sur le port 8420, pour votre programme.

Ce qu'il simule

La station A. Ses deux planches, korko-01 et korko-02, sont au râtelier. Les quatre autres, korko-03 à korko-06, appartiennent aux stations B et C : elles sont chez elles, donc hors de portée, et n'apparaissent pas dans le signal tant qu'on ne les rapporte pas ici.

La page de contrôle

Le graphe montre une courbe par balise : la puissance reçue en dBm, sur les quatre dernières minutes. Plus la planche est loin, plus la courbe descend : autour de -67 au râtelier, de -80 à cinq mètres, et plus rien en dessous de -100.

Le tableau des planches donne pour chacune la dernière puissance reçue, sa distance à la station et ce qu'elle est en train de faire.

Les curseurs règlent le bruit, l'exposant de propagation, le taux de perte de paquets et la vitesse du temps. Montez le bruit pour voir votre algorithme souffrir.

Trois modes

Manuel. Vous pilotez les planches vous-même, rien ne se déclenche tout seul. C'est le mode pour fabriquer un cas tordu à la demande.

Bouton	Ce qu'il fait	Vérité terrain
Part surfer	envoie la planche au large	un départ
Revient	la ramène au râtelier	un retour, à son arrivée
Sur le sable	la pose à neuf mètres	rien : ce n'est pas un départ
Au râtelier	la raccroche	rien
Corps devant	un corps mouillé entre balise et station, jusqu'à ce qu'on l'éteigne	rien
Planche à l'envers	face contre terre, environ cinq dB de moins	rien

Planche muette	coupe complètement l'émission	rien
Arrive ici	rapporte une planche de B ou C au râtelier A	une étrangère, à son arrivée
Repart chez elle	la renvoie hors de portée	rien

Scénarios. Sept situations prêtes à jouer, qui se déroulent toutes seules, et que l'on peut aussi lancer en ligne de commande, avec une note à la fin :

```
python3 ma_station.py --sim --scenario depart
```

Copier

Scénario	Ce qu'il joue
depart	un départ franc, puis rien
sable	une planche posée sur le sable, puis rangée : aucun départ
corps	un corps mouillé devant la balise, deux fois : aucun départ
morte	une balise qui faiblit, puis se tait
foule	deux départs rapprochés, deux retours
etrangere	une planche de la station B rapportée en A
journee	une matinée complète, avec tous les pièges ; c'est le scénario par défaut

Mes traces. Vos enregistrements, rejoués à l'identique. Aucune simulation : ce qui sort est exactement ce qui avait été observé la première fois.

Enregistrer un cas

Dans la barre du milieu, tapez un nom, cliquez sur **Enregistrer**, jouez votre scène, puis cliquez sur **Arrêter**. Deux fichiers arrivent dans le dossier `traces/` : les observations, et la vérité terrain qui va avec. Ils apparaissent aussitôt dans « Mes traces », et se rejouent aussi en ligne de commande :

```
python3 ma_station.py --rejeu traces/mon-essai.ndjson
```

Copier

C'est le bon réflexe quand vous tombez sur un cas qui met votre algorithme en défaut :
enregistrez-le une fois, rejouez-le autant que nécessaire

enregistrez le vrai rôle, rejouez le autant que nécessaire.

La vérité terrain, et le piège

Le simulateur sait ce qui s'est réellement passé : à tel instant, telle planche est vraiment partie. C'est ce qui permet de noter vos décisions.

Quatre situations font chuter le signal exactement comme un départ, sans en être un : la planche posée sur le sable, le corps devant la balise, la planche à l'envers, la balise muette. Et une situation fait monter le signal comme un retour sans en être un : une planche étrangère qui arrive. Pour elle, appelez `self.etrangere(...)` ; un `RETOUR` compterait comme une erreur, puisqu'elle n'est jamais partie d'ici. La liste des planches de chaque station est dans `korko.STATIONS` : c'est une table de configuration, jamais une déduction faite sur le signal.

Lire le score

En mode `--sim`, ou en `--rejeu` quand la vérité terrain est à côté des traces, vous obtenez par exemple, pour `station_exemple.py` sur la matinée complète :

```
détections justes ..... 4
faux départs ..... 7
faux retours ..... 7
événements manqués ..... 1
latence médiane ..... 6.8 s
```

[Copier](#)

Trois chiffres à faire descendre, un à surveiller. Sous quinze secondes de latence, c'est bien. L'exemple est fait pour être battu.

La façon de travailler qui marche

Réglez sur le simulateur, où vous fabriquez les cas difficiles à volonté. Validez sur des traces enregistrées, qui contiennent du vrai bruit. Prouvez sur la maquette, avec de vraies balises. Un algorithme parfait en simulation peut s'effondrer sur le signal réel : le simulateur produit un bruit plus propre que la vie.

Astuce : `python3 korko_test.py localhost --station A` vérifie aussi le simulateur, exactement comme une vraie station.

Ce qu'il y a dans le kit

Fichier	À quoi il sert
korko_sim.py	le simulateur : la station A, ses planches et les étrangères, une page de contrôle
korko.py	la bibliothèque commune, à ne pas modifier : lecture du flux, appel de vos méthodes, notation
korko_test.py	la vérification au vert, la recherche des stations (<code>--trouver</code>) et la mesure d'une balise (<code>--mesure korko-01</code>)
station_exemple.py	partie 1 : un exemple qui tourne, et ce qu'il fait mal
cloud_exemple.py	partie 2 : un exemple qui tourne, et ce qu'il fait mal

Chaque exemple se termine par la liste de ses faiblesses. Ce n'est pas un oubli : c'est le sujet.

[↓ korko-kit.zip](#)

Un problème d'installation ? Levez la main. Green Wave SAS — KORKO, Anglet.